

Chapter 9 Summary

Summary in Arabic

تلخيص شامل للفصل 9: تصميم قاعدة بيانات علائقية باستخدام التحويل من نموذج ER/EER إلى النماذج العلائقية

الهدف العام من التحويل

الهدف الأساسي من تحويل مخططات ER (Entity-Relationship) أو EER (Enhanced ER) إلى نماذج علائقية هو:

- الحفاظ على كل المعلومات (كل السمات، والعلاقات، والقيود).
- الحفاظ على القيود قدر الإمكان — مع العلم أن النموذج العلائقي لا يستطيع تمثيل بعض القيود مثل:
 - النسبة القصوى للارتباط (مثل 1:10 في ER).
 - التصنيف الشامل (مثلاً: كل طالب إما محلي أو أجنبي — ولا يمكن أن يكون كلاهما).
 - تقليل القيم الفارغة (Null Values) قدر المستطاع.
- هذا التحويل مُنفَّذ تلقائيًا في العديد من أدوات قواعد البيانات التجارية (مثل Oracle, SQL Server, MySQL, Workbench).

خوارزمية التحويل من ER إلى العلائقي (ER-to-Relational Mapping Algorithm)

الخطوة 1: تحويل الكيانات العادية (Regular Entity Types)

- لكل كيان عادي (قوي) في مخطط ER، ننشئ علاقة (جدول) واحدة.
- تحتوي هذه العلاقة على جميع السمات البسيطة للكيان.
- نختار أحد المفاتيح الرئيسية للكيان كمفتاح رئيسي للعلاقة.
- إذا كان المفتاح مركبًا (Composite Key)، فنستخدم جميع السمات البسيطة التي تكوّنهُ كمفتاح رئيسي مركب.

مثال: ✓

في مخطط شركة (COMPANY):

- كيان EMPLOYEE له سمات: SSN, Name, Address, Bdate → ننشئ جدول EMPLOYEE(SSN, Name, Address, Bdate) حيث SSN هو المفتاح الرئيسي.

- DEPARTMENT → DEPARTMENT(DNUMBER, DName, MgrSSN, MgrStartDate) كيان
- PROJECT → PROJECT(PNUMBER, PName, PLocation, DNUM) كيان

💡 ملاحظة مهمة: لا نحول السمات المركبة أو المتعددة القيم هنا — سنعالجها في خطوات لاحقة.

الخطوة 2: تحويل الكيانات الضعيفة (Weak Entity Types)

- الكيان الضعيف لا يملك مفتاحه الخاص، بل يعتمد على كيان آخر (يُسمى "مالك" أو Owner).
- ننشئ علاقة جديدة لل-ENTITY الضعيف.
- نضيف كعنصر في هذه العلاقة:
- جميع السمات البسيطة للكيان الضعيف.
- المفتاح الرئيسي للكيان المالك (يُصبح مفتاحًا أجنبيًا Foreign Key).
- المفتاح الرئيسي للعلاقة الجديدة هو:
- المفتاح الرئيسي للمالك + الجزء الجزئي (Partial Key) للكيان الضعيف.

✅ مثال:

- الكيان الضعيف DEPENDENT يعتمد على EMPLOYEE.
- سمات DEPENDENT : Dependent_name , Sex , Bdate , Relationship
- المفتاح الجزئي: Dependent_name
- المفتاح الرئيسي لـ EMPLOYEE : SSN
- ننشئ الجدول:
- DEPENDENT(Essn, Dependent_name, Sex, Bdate, Relationship)
- {Essn, Dependent_name} = المفتاح الرئيسي
- Essn هو مفتاح أجنبي يشير إلى EMPLOYEE(SSN)

⚠ ملاحظة: يجب أن يكون هناك ارتباط (relationship) بين الكيان الضعيف والمالك، وإلا لن يكون ضعيفًا!

الخطوة 3: تحويل العلاقات الثنائية 1:1 (Binary 1:1 Relationships)

لتحويل علاقة 1:1 بين كيانين A و B، لدينا ثلاث خيارات:

✅ الخيار 1: المفتاح الأجنبي (الأكثر شيوعًا)

- نختار أحد الكيانين (غالبًا الذي لديه مشاركة كاملة Total Participation) ونضيف مفتاحه الرئيسي كمفتاح أجنبي في جدول الكيان الآخر.
- مثال: العلاقة MANAGES بين EMPLOYEE و DEPARTMENT ، حيث كل إدارة لها مدير واحد فقط، وكل مدير يدير إدارة واحدة فقط (1:1)، وكل إدارة لديها مدير (مشاركة كاملة للإدارة).

نضيف المفتاح الرئيسي لـ EMPLOYEE (SSN) كمفتاح أجنبي في جدول DEPARTMENT ، ونسميه MgrSSN :

DEPARTMENT(DNumber, DName, MgrSSN, MgrStartDate)

-- MgrSSN → FK إلى EMPLOYEE(SSN)

الخيار 2: دمج الجدولين (Merged Relation) ✓

- ندمج كلا الكيانيين والعلاقة في جدول واحد.
- مناسب فقط عندما تكون المشاركة الكاملة في كلا الجانبين.
- مثال: إذا كانت العلاقة HAS_OFFICE بين EMPLOYEE و OFFICE (كل موظف له مكتب واحد، وكل مكتب لموظف واحد فقط)، يمكننا عمل جدول واحد:

EMPLOYEE_OFFICE(SSN, Name, OfficeID, Location, Floor)

الخيار 3: جدول منفصل للعلاقة (Cross-reference) ✓

- ننشئ جدولاً جديداً يحتوي فقط على المفتاحين الرئيسيين لكلا الكيانيين.
- نادر الاستخدام لأنه يزيد التعقيد دون فائدة حقيقية.

💡 **نصيحة:** استخدم الخيار 1 دائماً إلا إذا كنت متأكداً من أن الدمج أفضل (مثل حالات التصميم البسيطة).

الخطوة 4: تحويل العلاقات الثنائية 1:N (Binary 1:N Relationships)

- العلاقة 1:N تعني أن كيئناً واحداً (الجانب 1) يمكنه الارتباط بعدد غير محدود من الكيانات (الجانب N).
- القاعدة: نضيف المفتاح الرئيسي للكيان على الجانب 1 كمفتاح أجنبي في جدول الكيان على الجانب N.

مثال: ✓

العلاقة WORKS_FOR بين DEPARTMENT (1) و EMPLOYEE (N) :

- كل موظف يعمل في قسم واحد فقط.
 - كل قسم يمكن أن يضم عدة موظفين.
- نضيف DNUMBER (مفتاح DEPARTMENT) كمفتاح أجنبي في جدول EMPLOYEE ، ونسميه DNO :

EMPLOYEE(SSN, Name, DNO, ...)

-- DNO → FK إلى DEPARTMENT(DNUMBER)

🔴 **ملاحظة:** لا ننشئ جدولاً منفصلاً للعلاقة (إلا في حالات نادرة جداً). السمات المرتبطة بالعلاقة (مثل تاريخ بدء العمل) تُضاف مباشرة إلى جدول الجانب N.

الخطوة 5: تحويل العلاقات الثنائية M:N (Binary M:N Relationships)

- في العلاقات M:N، لا يمكننا وضع مفتاح أجنبي في أحد الجدولين فقط (لأن كل كيان يمكنه الارتباط بعدد كبير من الكيانات الأخرى).
- الحل: إنشاء جدول جديد يمثل العلاقة نفسها — يُسمى "جدول العلاقة" أو Relationship Table.

مثال: ✓

العلاقة WORKS_ON بين EMPLOYEE و PROJECT :

- موظف يمكنه العمل على أكثر من مشروع.
- مشروع يمكن أن يُنفذ بواسطة أكثر من موظف.
- ننشئ جدولاً جديداً:

```
WORKS_ON(Essn, Pno, Hours)
-- Essn → FK إلى EMPLOYEE(SSN)
-- Pno → FK إلى PROJECT(Pnumber)
-- (مرکّب) = {Essn, Pno} = المفتاح الرئيسي
-- Hours: سمة للعلاقة (عدد ساعات العمل)
```

✓ معلومة مهمة: أي سمة مرتبطة بالعلاقة (مثل Hours, StartDate) تُضاف مباشرة إلى هذا الجدول الجديد.

الخطوة 6: تحويل السمات المتعددة القيم (Multivalued Attributes)

- السمة المتعددة القيم (مثل: PhoneNumbers لموظف يمكن أن يكون لديه 3 أرقام هاتف) لا يمكن تخزينها في عمود واحد في جدول علائقي.
- الحل: إنشاء جدول منفصل لهذه السمة.

مثال: ✓

- كيان DEPARTMENT له سمة متعددة القيم: LOCATIONS (مواقع فرعية).
- ننشئ جدولاً جديداً:

```
DEPT_LOCATIONS(Dnumber, Dlocation)
-- Dnumber → FK إلى DEPARTMENT(DNUMBER)
-- = {Dnumber, Dlocation} = المفتاح الرئيسي
```

- الآن يمكن لقسم واحد أن يمتلك عدة مواقع:

(D2, Tokyo) , (D1, London) , (D1, NewYork)

💡 ملاحظة: حتى لو كانت السمة مركبة (مثل: Address(Street, City, Zip) ، نحول كل جزء بسيط منها إلى أعمدة منفصلة في الجدول الجديد.

الخطوة 7: تحويل العلاقات النونية (N-ary Relationships — N > 2)

- العلاقات التي تربط أكثر من كيانين (مثل 3 أو 4 كيانات) تُسمى N-ary.
- الحل: إنشاء جدول منفصل يحتوي على:
 - المفاتيح الرئيسية لجميع الكيانات المشاركة.
 - أي سمات مرتبطة بالعلاقة نفسها.

مثال: ✓

العلاقة SUPPLY بين PART , SUPPLIER , و PROJECT :

- مورد يزود قطعة لمشروع معين.
- ننشئ جدولاً:

```
SUPPLY(Sname, PartNo, ProjName, Quantity)
-- Sname → FK إلى SUPPLIER
-- PartNo → FK إلى PART
-- ProjName → FK إلى PROJECT
-- {Sname, PartNo, ProjName} = المفتاح الرئيسي
-- Quantity: سمة للعلاقة
```

♦ ملاحظة: لا يمكن تحويل هذه العلاقات باستخدام المفاتيح الأجنبية في الجداول الأصلية — لأن كل كيان قد يرتبط بأكثر من كيان آخر، وبالتالي لا يمكن تعيين مفتاح أجنبي واحد فقط.

تحويل مكونات نموذج EER (Enhanced ER) إلى علائقي

الخطوة 8: تحويل التخصص/التصنيف العام (/ Specialization Generalization)

لدينا كيان عام (Superclass) وعدد من الكيانات المتخصصة (Subclasses). لدينا 4 خيارات:

الخيار 8A: جداول منفصلة لل superclass و subclasses ✓

- ننشئ جدولاً للكيان العام (Superclass).
- وجداولاً منفصلاً لكل subclass.
- كل جدول فرعي يحتوي على المفتاح الرئيسي لل superclass (كـ FK) + سماته الخاصة.

مثال:

- Superclass: EMPLOYEE(SSN, Name, Salary, JobType)
- Subclasses: ENGINEER , MANAGER , SALESREP

```
EMPLOYEE(SSN, Name, Salary, JobType) -- المفتاح : SSN
ENGINEER(SSN, EngineeringField) -- أيضًا PK و FK هو SSN
MANAGER(SSN, DepartmentManaged) -- أيضًا PK و FK هو SSN
```

✓ **مميزاته:** يدعم أي نوع من التخصص (كامل/جزئي، منفصل/متداخل).

✗ **عيوبه:** يحتاج إلى عمليات JOIN عند استرجاع بيانات كاملة.

✓ الخيار 8B: جداول فقط لل subclasses (بدون superclass)

- ندمج سمات الكيان العام مع كل subclass.
- لا ننشئ جدولاً لل superclass.
- **شرط أساسي:** التخصص يجب أن يكون كاملاً (Total) — أي كل كيان في superclass موجود في أحد subclasses على الأقل.

مثال:

- Superclass: VEHICLE(PlateNum, Make, Model, Year)
- Subclasses: CAR, TRUCK

```
CAR(PlateNum, Make, Model, Year, NumDoors)
TRUCK(PlateNum, Make, Model, Year, MaxLoad)
```

✓ مناسب جدًا للحالات التي تكون فيها التخصصات شاملة.

✗ لا يناسب التخصصات الجزئية (إذا كان هناك سيارة لا تنتمي لأي subclass!).

✓ الخيار 8C: جدول واحد مع سمة نوع (Type Attribute)

- ننشئ جدولاً واحداً يجمع كل السمات من superclass وجميع subclasses.
- نضيف سمة واحدة تسمى Type أو JobType لتحديد ما إذا كان السجل subclass 屬於 معين.

مثال:

```
EMPLOYEE(SSN, Name, Salary, JobType, EngineeringField, DepartmentManaged,
SalesRegion)
-- JobType: 'Engineer', 'Manager', 'SalesRep'
-- NULL سيكون له قيمة، والباقي EngineeringField إذا كان الموظف مهندساً، فإن
```

✓ بسيط، لا يحتاج JOIN.

✗ كثير من القيم الفارغة (NULLs) — غير فعال من حيث التخزين.

✓ الخيار 8D: جدول واحد مع سمات نوع منطقية (Boolean Flags)

- نستخدم أعمدة منطقية (BOOLEAN) لكل subclass.
- كل عمود يشير إلى ما إذا كان السجل ينتمي لذلك subclass.

```
EMPLOYEE(SSN, Name, Salary, IsEngineer, IsManager, IsSalesRep,
EngineeringField, DepartmentManaged, SalesRegion)
```

```
-- IsEngineer = TRUE → يدل على أنه مهندس
```

```
-- IsManager = FALSE → ليس مديرًا
```

✓ مفيد جدًا في الاستعلامات السريعة (مثل: "أظهر كل المديرين").

✗ يتطلب صيانة دقيقة لضمان عدم تكرار التصنيف (مثلاً: لا يمكن أن يكون شخص مهندس ومدير في نفس الوقت إلا إذا كان التخصص متداخلًا).

✦ ملاحظة مهمة:

- الخيارات A و B تفضل في التطبيقات التي تتطلب التكامل والقيود.
- الخيارات C و D تفضل في التطبيقات التي تحتاج سرعة استعلام (مثل تقارير BI) رغم وجود Nulls.

الخطوة 8 الإضافية: التخصص المشترك (Shared Subclass / Multiple Inheritance)

- عندما يكون هناك subclass يرث من أكثر من كيان عام (مثل: STUDENT_ASSISTANT يرث من STUDENT و EMPLOYEE).
- شرط أساسي: يجب أن يكون لدى الكيانين العامين نفس المفتاح الرئيسي (مثل: SSN).
- نطبق أحد الخيارات السابقة (8A, 8C, 8D) على هذا subclass.

✓ مثال:

- STUDENT(SSN, Major, GPA)
 - EMPLOYEE(SSN, Dept, Salary)
 - STUDENT_ASSISTANT ← يرث من كليهما
- C: نستخدم الخيار 8 →

```
STUDENT_ASSISTANT(SSN, Major, GPA, Dept, Salary, Role)
```

```
-- Role: 'TA' أو 'RA'
```

أو الخيار 8A: جدول منفصل يحتوي على SSN فقط (كمفتاح رئيسي وأجنبي) + سماته الخاصة.

الخطوة 9: تحويل أنواع الاتحاد (Union Types / Categories)

- الوحدة (Category) هي مجموعة من الكيانات المختلفة (ليس لها نفس المفتاح!) تشارك في علاقة واحدة.

- مثال: OWNER يمكن أن يكون إما PERSON أو CORPORATION — ولكل منهما مفتاح مختلف (SSN vs TaxID).

✓ الحل: استخدام مفتاح اصطناعي (Surrogate Key)

- ننشئ جدولاً جديداً للوحدة (Category).
- نضيف عموداً جديداً كمفتاح رئيسي اصطناعي (مثل: OwnerID) — رقم تسلسلي لا يحمل معنى.
- نربط هذا المفتاح الاصطناعي بالكيانات الأصلية عبر مفاتيح أجنبية.

✓ مثال:

```
OWNER(OwnerId, Type) -- Type: 'Person' أو 'Corp'
PERSON(OwnerId, SSN, Name, Phone) -- OwnerId → FK إلى OWNER
CORPORATION(OwnerId, TaxID, Name, Addr) -- OwnerId → FK إلى OWNER
```

- الآن، يمكن لـ OWNER أن يشير إلى أي من الشخص أو الشركة عبر OwnerId.

✓ هذا الحل يحل مشكلة اختلاف المفاتيح بين الكيانات المتنوعة.
✗ بدونها، لا يمكن تمثيل الوحدة في النموذج العلائقي.

ملخص سريع للخوارزمية (جدول تلخيصي)

الخطوة	المكون	التحويل
1	كيان عادي	جدول واحد + سماته + مفتاحه الرئيسي
2	كيان ضعيف	جدول جديد + مفتاح المالك كـ FK + المفتاح الجزئي كجزء من PK
3	علاقة 1:1	اختيار: (أ) مفتاح أجنبي في أحد الجدولين، (ب) دمج، (ج) جدول منفصل
4	علاقة N:1	إضافة المفتاح الرئيسي للجانب 1 كـ FK في جدول الجانب N
5	علاقة M:N	جدول منفصل يحتوي على مفاتيح كلا الكيانيين + سمات العلاقة
6	سمة متعددة القيم	جدول منفصل: المفتاح الأجنبي + السمة + PK مركب
7	علاقة N-ary (N>2)	جدول منفصل: مفاتيح جميع الكيانات + سمات العلاقة
8	تخصص/تصنيف عام	4 خيارات: A (جداول منفصلة)، B (بدون superclass)، C (سمة نوع)، D (فلات منطقية)
9	وحدة (Category)	جدول جديد + مفتاح اصطناعي (Surrogate Key) يربط بالكيانات المختلفة

ملاحظات ختامية مهمة

1. المفتاح الاصطناعي (Surrogate Key):

- غالبًا ما يستخدم في الحالات التي لا يوجد فيها مفتاح طبيعي مناسب (مثل: في الوحدات، أو عندما يكون المفتاح الطبيعي طويلًا أو غير مستقر).
- مثال: EmployeeID INT AUTO_INCREMENT بدلاً من SSN.

2. القيم الفارغة (NULLs):

- لا يمكن تجنبها تمامًا — خاصة في الخيارات C و D.
- لكن يمكن تقليلها باختيار الخيارات المناسبة (مثل A أو B).

3. القيود غير المدعومة:

- لا يمكن تمثيل قيود مثل: "لا يمكن أن يكون الموظف مهندسًا ومديرًا في نفس الوقت" إلا باستخدام تطبيق تطوري (Application Logic) أو قيود (Triggers) في قاعدة البيانات.

4. التحول التلقائي:

- معظم أدوات التصميم (مثل ERwin, Lucidchart, MySQL Workbench) تقوم بهذا التحويل تلقائيًا — لكن من المهم فهمه لاختيار الأنسب!

5. الأفضلية العملية:

- في المشاريع الكبيرة: استخدم الخيار 8A (جداول منفصلة) للحفاظ على التنظيم والقيود.
- في أنظمة التقارير: استخدم الخيار 8C أو 8D لسهولة الاستعلام.

خلاصة نهائية

التحول من ER/EER إلى العلائقي ليس مجرد عملية تقنية، بل هو قرار تصميمي استراتيجي. الاختيار بين الخيارات (خاصة في التخصص والوحدات) يؤثر على:

- أداء النظام.
- سهولة الصيانة.
- دقة البيانات.
- قابلية التوسع.

فهم هذه الخطوات يمنحك القدرة على:

- تحليل أي مخطط ER/EER وتحويله بدقة.
- اتخاذ قرارات تصميمية مدروسة.
- تجنب الأخطاء الشائعة (مثل فقدان البيانات أو تكرارها).

Summary in English

Comprehensive Summary of Chapter 9: Relational Database Design by ER- and EER-to-

Relational Mapping

Overall Goal of the Mapping Process

The primary objectives when transforming an **ER (Entity-Relationship)** or **EER (Enhanced ER)** schema into a relational schema are:

- **Preserve all information** — every attribute, relationship, and constraint must be captured.
- **Maintain constraints as much as possible** — recognize that the relational model cannot enforce certain constraints found in ER/EER models, such as:
 - Maximum cardinality ratios (e.g., 1:10).
 - Exhaustive classification (e.g., “Every STUDENT is either Domestic or Foreign — no other option”).
- **Minimize null values** — avoid unnecessary NULLs in relations where possible.

💡 This mapping procedure has been automated in many commercial database design tools (e.g., Oracle SQL Developer, MySQL Workbench, ERwin).

ER-to-Relational Mapping Algorithm

Step 1: Mapping Regular (Strong) Entity Types

- For each regular (strong) entity type **E** in the ER schema, create one relation (table) **R**.
- Include **all simple attributes** of **E** as attributes of **R**.
- Select **one key attribute** of **E** as the **primary key (PK)** of **R**.
- If the chosen key is **composite**, then **all its simple components** together form the composite primary key of **R**.

✅ Example:

From the COMPANY ER diagram:

- Entity **EMPLOYEE** → Relation: **EMPLOYEE(SSN, Name, Address, Bdate)**
→ PK = **SSN**
- Entity **DEPARTMENT** → Relation: **DEPARTMENT(DNUMBER, DName, MgrSSN, MgrStartDate)**
→ PK = **DNUMBER**
- Entity **PROJECT** → Relation: **PROJECT(PNUMBER, PName, PLocation, DNUM)**
→ PK = **PNUMBER**

💡 Note: Composite attributes (e.g., **Address(Street, City, State)**) are broken down into individual simple attributes during this step. Multivalued attributes are handled separately in Step 6.

Step 2: Mapping Weak Entity Types

- A weak entity does not have its own key; it depends on a **strong owner entity**.
- Create a new relation `R` for the weak entity type `W`.
- Include:
 - All **simple attributes** of `W`.
 - The **primary key(s)** of the **owner entity type(s)** as **foreign key(s)** in `R`.
- The **primary key of** `R` is the combination of:
 - The primary key(s) of the owner entity(ies), and
 - The **partial key** of the weak entity (if any).

✓ Example:

- Weak entity: `DEPENDENT`, owned by `EMPLOYEE`.
- Attributes of `DEPENDENT`: `Dependent_name`, `Sex`, `Bdate`, `Relationship`
- Partial key: `Dependent_name`
- Owner's PK: `SSN` from `EMPLOYEE`
- Resulting relation:

```
DEPENDENT(Essn, Dependent_name, Sex, Bdate, Relationship)
-- PK = {Essn, Dependent_name}
-- Essn is FK referencing EMPLOYEE(SSN)
```

⚠ **Critical:** Without the owner entity and its key, the entity is not weak. Always verify the identifying relationship.

Step 3: Mapping Binary 1:1 Relationships

For a binary 1:1 relationship `R` between two entities `S` and `T`, you have **three options**:

✓ Option 1: Foreign Key Approach (Recommended)

- Add the **primary key of one entity** as a **foreign key** in the **other entity's relation**.
- Prefer to place the foreign key in the relation representing the entity with **total participation** in the relationship.

✓ Example:

- Relationship: `MANAGES` between `EMPLOYEE` and `DEPARTMENT`
- Total participation: Every department has exactly one manager → `DEPARTMENT` has total participation.

- Solution: Add `MgrSSN` (FK from `EMPLOYEE.SSN`) to `DEPARTMENT` :

```
DEPARTMENT(DNUMBER, DName, MgrSSN, MgrStartDate)
-- MgrSSN → FK → EMPLOYEE(SSN)
```

✓ Option 2: Merged Relation (Single Table)

- Merge both entities and the relationship into **one single relation**.
- Only appropriate if **both participations are total**.

✓ Example:

- Relationship: `HAS_OFFICE` between `EMPLOYEE` and `OFFICE` (each employee has one office, each office assigned to one employee).

```
EMPLOYEE_OFFICE(SSN, Name, OfficeID, Location, Floor)
-- PK = SSN or OfficeID (both unique)
```

✓ Option 3: Cross-Reference Relation (Three Relations)

- Create a **third relation** containing only the two primary keys.
- Rarely used — adds complexity without benefit unless needed for specific integrity rules.

💡 **Best Practice:** Use **Option 1** unless merging simplifies queries significantly and both sides have total participation.

Step 4: Mapping Binary 1:N Relationships

- In a 1:N relationship, the “N” side can have multiple instances linked to one instance on the “1” side.
- **Rule:** Add the **primary key of the “1” entity** as a **foreign key** in the **relation representing the “N” entity**.
- Include any **simple attributes of the relationship** as additional attributes in the “N” side relation.

✓ Example:

- Relationship: `WORKS_FOR` between `DEPARTMENT` (1) and `EMPLOYEE` (N)
- Each employee works for one department; a department can have many employees.
→ Add `DNUMBER` (from `DEPARTMENT`) as `DNO` (FK) in `EMPLOYEE` :

```
EMPLOYEE(SSN, Name, DNO, ...)
```

📌 **Note:** Do NOT create a separate relation for 1:N relationships unless there's a compelling reason (rare).

Step 5: Mapping Binary M:N Relationships

- M:N relationships cannot be represented by adding a foreign key to either side.
- **Solution:** Create a **new relation (relationship table)** to represent the relationship itself.
- Include:
 - The **primary keys** of **both participating entities** as **foreign keys**.
 - The **combination** of these two foreign keys becomes the **composite primary key** of the new relation.
 - Any **simple attributes** of the relationship are included as regular attributes.

✅ Example:

- Relationship: WORKS_ON between EMPLOYEE and PROJECT
- An employee can work on many projects; a project can have many employees.

```
WORKS_ON(Essn, Pno, Hours)
-- Essn → FK → EMPLOYEE(SSN)
-- Pno → FK → PROJECT(Pnumber)
-- PK = {Essn, Pno}
-- Hours: attribute of the relationship
```

✅ This is the **only correct way** to map M:N relationships in the relational model.

Step 6: Mapping Multivalued Attributes

- A multivalued attribute (e.g., PhoneNumbers for an employee) cannot be stored directly in a single column in a relational table.
- **Solution:** Create a **separate relation** for the multivalued attribute.
- Include:
 - The **primary key** of the entity/relationship that owns the multivalued attribute (as a foreign key).
 - The **multivalued attribute itself** (or its simple components if composite).
- The **primary key** of the new relation is the **combination** of the foreign key and the multivalued attribute.

✓ Example:

- Entity: DEPARTMENT has multivalued attribute LOCATIONS

```
DEPT_LOCATIONS(Dnumber, Dlocation)
-- Dnumber → FK → DEPARTMENT(DNUMBER)
-- PK = {Dnumber, Dlocation}
```

Now, one department can have multiple locations:

- (D1, New York)
- (D1, London)
- (D2, Tokyo)

💡 Even if the attribute is composite (e.g., Address(Street, City, Zip)), break it into separate columns within this new relation.

Step 7: Mapping N-ary Relationships ($N > 2$)

- For relationships involving more than two entities (ternary, quaternary, etc.), create a **dedicated relationship relation**.
- Include:
 - The **primary keys** of **all participating entity relations** as foreign keys.
 - Any **simple attributes** of the N-ary relationship.
- The **primary key** of the new relation is the **combination of all foreign keys**.

✓ Example:

- Relationship: SUPPLY connecting SUPPLIER, PART, and PROJECT

```
SUPPLY(Sname, PartNo, ProjName, Quantity)
-- Sname → FK → SUPPLIER
-- PartNo → FK → PART
-- ProjName → FK → PROJECT
-- PK = {Sname, PartNo, ProjName}
-- Quantity: attribute of the relationship
```

🔍 **Important:** You **cannot** represent ternary relationships using foreign keys in existing tables — the relational model requires a separate junction table.

Mapping EER Model Constructs to Relations

Step 8: Options for Mapping Specialization/Generalization Hierarchies

Convert a specialization/generalization hierarchy with superclass C and m subclasses $\{S_1, S_2, \dots, S_m\}$ using **one of four options**:

✓ Option 8A: Multiple Relations — Superclass + Subclasses

- Create one relation for the **superclass** C .
- Create one relation for **each subclass** S_i .
- Each subclass relation includes:
 - Its own attributes.
 - The **primary key of the superclass** (which also serves as its own primary key — a **foreign key**).
- Works for **any** specialization: total/partial, disjoint/overlapping.

✓ Example (Figure 4.4):

```
EMPLOYEE(SSN, Name, Salary, JobType)  -- Superclass
ENGINEER(SSN, EngineeringField)      -- Subclass: PK=SSN, FK to EMPLOYEE
MANAGER(SSN, DepartmentManaged)     -- Subclass: PK=SSN, FK to EMPLOYEE
SALESREP(SSN, SalesRegion)           -- Subclass: PK=SSN, FK to EMPLOYEE
```

- ✓ Pros: Full constraint preservation, supports overlapping and partial specializations.
- ✗ Cons: Requires JOINS to retrieve full data.

✓ Option 8B: Multiple Relations — Subclasses Only

- **Do not create a superclass relation.**
- Create **only** relations for each subclass.
- Each subclass relation includes **all attributes** of the superclass **plus** its own attributes.
- **Only valid if the specialization is TOTAL** (every instance of the superclass belongs to at least one subclass).

✓ Example (Figure 4.3b: VEHICLE → CAR, TRUCK):

```
CAR(PlateNum, Make, Model, Year, NumDoors)
TRUCK(PlateNum, Make, Model, Year, MaxLoad)
```

- ✓ Pros: No redundancy, avoids NULLs.
- ✗ Cons: Cannot represent partial specializations. If a vehicle doesn't fit any subclass, it's lost.

✓ Option 8C: Single Relation with One Type Attribute

- Create **one single relation** combining **all attributes** of the superclass and all subclasses.
- Add a **type attribute** (*t*) that indicates which subclass the tuple belongs to.

✓ Example (Figure 4.4):

```
EMPLOYEE(SSN, Name, Salary, JobType, EngineeringField, DepartmentManaged,
SalesRegion)
-- JobType = 'Engineer', 'Manager', 'SalesRep'
-- Only one of the three fields will have a value per row; others are NULL
```

✓ Pros: Simple queries, no JOINS needed.

✗ Cons: Many NULLs — inefficient storage and potential for inconsistent data.

✓ Option 8D: Single Relation with Multiple Boolean Type Attributes

- Create **one single relation** with all attributes.
- Add **one Boolean flag** per subclass: *IsEngineer*, *IsManager*, etc.
- Each flag indicates whether the tuple belongs to that subclass.

✓ Example (Figure 4.5 — Overlapping Specialization: Manager & Programmer):

```
EMPLOYEE(SSN, Name, Salary, Mflag, Pflag, DeptManaged, ProgrammingLanguage)
-- Mflag = TRUE → is a Manager
-- Pflag = TRUE → is a Programmer
-- Can be both!
```

✓ Pros: Supports overlapping specializations easily. Fast filtering: *WHERE Mflag = TRUE*.

✗ Cons: More complex integrity enforcement — must ensure flags are consistent (e.g., no row has all flags FALSE).

✦ Recommendation:

- Use **Option 8A** for maximum flexibility and constraint support.
- Use **Option 8C/D** for reporting/analytics systems where query speed matters more than normalization.

Step 8 (Additional): Mapping Shared Subclasses (Multiple Inheritance)

- A shared subclass inherits from **two or more superclasses**.

- **Constraint:** All parent classes must share the **same primary key attribute** (otherwise, use a category — Step 9).
- Apply **any of the above four options (8A–8D)** to the shared subclass, under the same constraints.

✓ **Example:** STUDENT_ASSISTANT inherits from STUDENT and EMPLOYEE (both use SSN)

Using **Option 8C:**

```
STUDENT_ASSISTANT(SSN, Major, GPA, Dept, Salary, Role)
-- Role: 'TA' or 'RA'
```

Using **Option 8A:**

```
STUDENT(SSN, Major, GPA)
EMPLOYEE(SSN, Dept, Salary)
STUDENT_ASSISTANT(SSN, Role)
-- SSN is PK and FK in all three
```

- ✓ Ensures inheritance while maintaining referential integrity.

Step 9: Mapping Union Types (Categories)

- A **category** (union type) represents a set of entities from **different entity types** that participate in a common relationship.
- The key problem: **Different entity types have different primary keys** (e.g., PERSON(SSN) vs CORPORATION(TaxID)).

✓ **Solution:** Introduce a **Surrogate Key**

- Create a **new relation** for the category.
- Add a **new artificial (surrogate) primary key** (e.g., OwnerId).
- Link this surrogate key to each contributing entity via **foreign keys**.

✓ **Example (Figure 4.8: OWNER category):**

```
OWNER(OwnerId, Type)                -- Type: 'Person' or 'Corp'
PERSON(OwnerId, SSN, Name, Phone)   -- OwnerId → FK → OWNER
CORPORATION(OwnerId, TaxID, Name, Addr) -- OwnerId → FK → OWNER
```

- Now, OWNER can reference either a person or a corporation.

- `OwnerId` is the **surrogate key** — it has no business meaning.

✖ This is the **only reliable way** to model categories in the relational model. Without it, you cannot unify entities with incompatible keys.

Summary Table: ER/EER to Relational Mapping Rules

Step	Construct	Mapping Rule
1	Regular Entity	One relation per entity; all simple attributes; one key as PK
2	Weak Entity	One relation; include owner's PK as FK + partial key → composite PK
3	Binary 1:1	Choose: (A) FK in one relation (prefer total participation), (B) merge, (C) cross-reference
4	Binary 1:N	Add PK of "1" side as FK in "N" side relation
5	Binary M:N	Create new relation with both PKs as FKs → composite PK
6	Multivalued Attribute	New relation: FK + attribute → composite PK
7	N-ary Relationship (N>2)	New relation: FKs of all participating entities → composite PK
8A	Generalization/Specialization	Separate relations for superclass + each subclass (recommended)
8B	Total Specialization	Only subclass relations; include superclass attributes
8C	Single Relation w/ Type Attr	One relation; type attribute indicates subclass
8D	Single Relation w/ Boolean Flags	One relation; Boolean flags for each subclass
9	Category (Union Type)	New relation with surrogate key; link to all contributing entities

Key Notes & Best Practices

1. Surrogate Keys:

- Essential for categories (Step 9) and sometimes preferred even for regular entities (e.g., `EmployeeID INT AUTO_INCREMENT` instead of `SSN`).
- Avoids issues with long, non-numeric, or changing natural keys.

2. Null Values:

- Unavoidable in Options 8C and 8D.
- Minimize them by choosing Option 8A or 8B where possible.

3. Constraints Not Enforceable:

- ER constraints like “max cardinality 1:10”, “exhaustive classification”, or “disjointness” **cannot be enforced by the relational model alone.**
- These must be enforced via **application logic, triggers, or stored procedures.**

4. Tool Automation:

- Tools like ERwin, Lucidchart, and MySQL Workbench auto-generate relational schemas.
- But understanding the underlying logic allows you to **review, optimize, and debug** the output.

5. Practical Recommendation:

- **For OLTP systems (transactional):** Use **Option 8A** — better normalization, fewer NULLs, easier integrity.
- **For Data Warehouses / BI Reports:** Use **Option 8C or 8D** — faster queries, fewer JOINS.

Final Summary

Transforming ER/EER to relational is not mechanical — it’s a design decision.

Your choice among the options affects:

- **Performance** (JOINS vs flat tables),
- **Data Integrity** (NULLs, consistency),
- **Maintainability** (clarity of structure),
- **Scalability** (future changes).

By mastering these nine steps, you gain the ability to:

- Accurately translate any conceptual ER/EER model into a robust relational schema.
- Avoid common pitfalls (lost data, redundant NULLs, unenforceable constraints).
- Communicate effectively with developers and DBAs about schema design trade-offs.